ESPECIFICACIONES DE EVALUACIÓN TÉCNICA

Desarrollo Full Stack de Plataforma de Solicitudes Internas y Notificaciones

Área responsable	**Departamento de Tecnología de la Información**
Unidad	División de Canales de Atención
Tipo de evaluación	Prueba técnica Full Stack
Stack sugerido	.NET 8, React + TypeScript, SQL Server u Oracle
Modalidad	Proyecto práctico más preguntas de conceptualización

Documento base para evaluar competencias técnicas, criterio de diseño, capacidad de integración y buenas prácticas de ingeniería de software.

Uso exclusivo de la Superintendencia de Bancos. Distribución autorizada por el área responsable.

Tabla de contenido

1. Objetivo
2. Alcance
3. Contexto del ejercicio
4. Desarrollo de aplicación web de gestión de solicitudes internas
5. Requisitos funcionales
6. Requisitos no funcionales
7. Requisitos técnicos
8. Modelo de datos mínimo esperado
9. Servicios/API sugeridos
10. Criterios de evaluación
11. Preguntas de conceptualización
12. Forma de entrega

Uso exclusivo de la Superintendencia de Bancos. Distribución autorizada por el área responsable.

1. Objetivo

Evaluar candidatos Full Stack mediante un ejercicio práctico orientado a la construcción de una solución empresarial mantenible, segura y trazable. La prueba busca validar dominio técnico en C# .NET, React, bases de datos relacionales, integración frontend-backend, modelado de información y aplicación de buenas prácticas de desarrollo.

2. Alcance

La evaluación contempla la implementación de una aplicación web para registrar, gestionar, clasificar y dar seguimiento a solicitudes internas de servicios tecnológicos, incluyendo notificaciones, estados, prioridades, comentarios y reportes operativos.

- Diseño e implementación de una API RESTful con .NET 8.
- Construcción de una interfaz web con React y TypeScript.
- Persistencia en SQL Server u Oracle, con modelo relacional documentado.
- Autenticación y autorización básica mediante JWT y roles.
- Aplicación de arquitectura limpia, separación de responsabilidades y principios SOLID.
- Registro de eventos relevantes para trazabilidad y auditoría técnica.
- Pruebas unitarias o de integración sobre componentes críticos.

La prueba permite evaluar no solo la funcionalidad final, sino también la calidad de la solución, la claridad del código, el criterio de diseño y la capacidad del candidato para construir software extensible en un contexto empresarial.

Uso exclusivo de la Superintendencia de Bancos. Distribución autorizada por el área responsable.

3. Contexto del ejercicio

La organización requiere una plataforma interna que permita a los colaboradores registrar solicitudes de soporte o requerimientos tecnológicos. Cada solicitud debe pasar por diferentes estados, puede estar asociada a un área, tener prioridad, ser asignada a un analista y generar notificaciones cuando ocurran cambios relevantes.

El candidato debe proponer e implementar una solución funcional, evitando acoplamientos innecesarios y dejando evidencia de decisiones técnicas importantes en el README del proyecto.

4. Desarrollo de aplicación web de gestión de solicitudes internas

Enunciado:

Desarrollar una aplicación web que permita registrar solicitudes internas, consultar su historial, gestionar cambios de estado, asignar responsables, emitir notificaciones y visualizar indicadores básicos de gestión.

4.1 La solución debe permitir

- Registrar solicitudes con título, descripción, tipo, prioridad, área solicitante y usuario solicitante.
- Consultar solicitudes mediante filtros por estado, prioridad, área, solicitante, responsable y rango de fechas.
- Cambiar el estado de una solicitud manteniendo historial de transición.
- Asignar o reasignar solicitudes a un responsable técnico.
- Agregar comentarios internos o públicos asociados a la solicitud.
- Enviar o simular notificaciones ante creación, asignación, cambio de estado y cierre.
- Gestionar usuarios con roles mínimos: Administrador, Analista y Solicitante.
- Mostrar un tablero con métricas básicas de solicitudes abiertas, cerradas, vencidas y por prioridad.

5. Requisitos funcionales

Gestión de solicitudes

- Crear, editar parcialmente y consultar solicitudes.
- Validar campos obligatorios y longitud máxima de textos principales.
- Permitir adjuntar una referencia textual o URL de evidencia, sin requerir almacenamiento físico de archivos.
- Generar un código único legible para cada solicitud, por ejemplo SOL-2026-0001.

Flujo de estados

- Estados mínimos: Registrada, En análisis, En progreso, En espera del solicitante, Resuelta y Cerrada.
- No permitir cerrar una solicitud que no tenga comentario de resolución.
- Registrar fecha, usuario y comentario en cada transición de estado.
- Permitir reabrir una solicitud cerrada solo para usuarios con rol Administrador o Analista.

Notificaciones

- Generar una notificación cuando se cree una solicitud.
- Generar una notificación cuando la solicitud sea asignada o cambie de estado.
- La notificación puede implementarse como registro en base de datos, simulación en consola, correo simulado o evento en cola.

Uso exclusivo de la Superintendencia de Bancos. Distribución autorizada por el área responsable.

- La solución debe permitir extender el mecanismo de notificación sin modificar directamente la lógica de negocio principal.

Seguridad y roles

- Autenticación mediante JWT.
- Administrador: acceso completo a catálogos, usuarios y solicitudes.
- Analista: acceso a solicitudes asignadas o disponibles para gestión.
- Solicitante: acceso a creación y seguimiento de sus propias solicitudes.

Reportes y tablero

- Cantidad de solicitudes por estado.
- Cantidad de solicitudes por prioridad.
- Solicitudes vencidas según fecha compromiso.
- Últimas solicitudes registradas con estado actual y responsable.

6. Requisitos no funcionales

Categoría	Requisito	Evidencia esperada
Usabilidad	Interfaz clara, formularios con validaciones visibles y navegación consistente.	Capturas, flujo funcional o explicación en README.
Mantenibilidad	Separación por capas o módulos, responsabilidades claras y bajo acoplamiento.	Estructura del proyecto y justificación técnica.
Escalabilidad	Capacidad de agregar nuevos tipos de solicitud, estados o mecanismos de notificación.	Uso de interfaces, servicios y patrones extensibles.
Rendimiento	Listados paginados y filtros aplicados desde backend.	Endpoint con paginación y consultas optimizadas.
Trazabilidad	Registro de cambios de estado, usuario y fecha.	Tabla de historial y consulta visible.
Testabilidad	Lógica de negocio desacoplada de infraestructura.	Pruebas unitarias o de integración reproducibles.
Seguridad	Validaciones de autorización en backend, no solo en frontend.	Uso de policies, roles o validaciones equivalentes.

Uso exclusivo de la Superintendencia de Bancos. Distribución autorizada por el área responsable.

7. Requisitos técnicos

- Backend: .NET 8, C#, ASP.NET Core Web API y Entity Framework Core.
- Frontend: React con TypeScript, componentes reutilizables y manejo ordenado de estado.
- Base de datos: SQL Server u Oracle. Debe incluir script de creación y datos de ejemplo o migraciones de EF Core.
- Autenticación: JWT con roles mínimos definidos.
- Documentación de API: Swagger/OpenAPI habilitado para ejecución local.
- Arquitectura: solución modular, Clean Architecture, arquitectura por capas o enfoque equivalente debidamente justificado.
- Control de versiones: repositorio Git con historial claro de commits.
- Logging: registro de errores y eventos principales de negocio.
- Configuración: uso de appsettings o variables de entorno para cadenas de conexión y parámetros sensibles.

Notas visuales sugeridas:

- Usar una paleta sobria basada en azul institucional y grises neutros.
- Evitar interfaces sobrecargadas. Priorizar legibilidad, contraste y consistencia.
- El diseño no necesita ser idéntico a ningún portal existente, pero debe reflejar criterio profesional.

8. Modelo de datos mínimo esperado

El candidato puede extender el modelo, pero la solución debe contemplar al menos las siguientes entidades o equivalentes:

Entidad	Campos mínimos	Relaciones esperadas	Observaciones
Usuario	Id, nombre, correo, rol, activo	Solicitudes creadas, solicitudes asignadas, comentarios	Puede implementarse con Identity o modelo propio simplificado.
Solicitud	Id, código, título, descripción, prioridad, estado, fechaCreación, fechaCompromiso	Usuario solicitante, responsable, área, tipo	Entidad principal del dominio.
Área	Id, nombre, activa	Solicitudes	Catálogo administrable o semilla inicial.
TipoSolicitud	Id, nombre, descripción, activo	Solicitudes	Permite clasificar requerimientos.
HistorialEstado	Id, estadoAnterior, estadoNuevo, fecha, comentario	Solicitud, usuario	Base para trazabilidad y auditoría.
Comentario	Id, texto, visibilidad, fecha	Solicitud, usuario	Diferenciar comentarios internos y visibles al solicitante.
Notificación	Id, canal, asunto, mensaje, estado, fecha	Solicitud, usuario destino	Puede ser simulada o persistida.

9. Servicios/API sugeridos

Método	Ruta sugerida	Propósito
POST	/api/auth/login	Autenticación y emisión de token JWT.
GET	/api/solicitudes	Consulta paginada con filtros.
POST	/api/solicitudes	Creación de solicitud.
GET	/api/solicitudes/{id}	Detalle de solicitud con historial y comentarios.
PATCH	/api/solicitudes/{id}/estado	Cambio de estado con comentario obligatorio según el caso.

Uso exclusivo de la Superintendencia de Bancos. Distribución autorizada por el área responsable.

PATCH	/api/solicitudes/{id}/asignacion	Asignación o reasignación de responsable.
POST	/api/solicitudes/{id}/comentarios	Registro de comentario interno o público.
GET	/api/dashboard/resumen	Métricas básicas del tablero.
GET	/api/catalogos/areas	Consulta de áreas activas.
GET	/api/catalogos/tipos-solicitud	Consulta de tipos de solicitud activos.

10. Criterios de evaluación

Criterio	Peso sugerido	Aspectos observables
Funcionalidad implementada	20%	Cumplimiento de flujo principal, filtros, estados, asignación y tablero.
Diseño backend y arquitectura	20%	Separación de capas, servicios, entidades, contratos, validaciones y manejo de errores.
Frontend y experiencia de usuario	15%	Componentes claros, formularios, navegación, manejo de estados y mensajes al usuario.
Base de datos y consultas	15%	Modelo relacional, integridad, scripts, migraciones, paginación y filtros eficientes.
Seguridad y trazabilidad	10%	JWT, roles, autorización backend, historial y logging.
Pruebas y calidad de código	10%	Pruebas automatizadas, legibilidad, nombres claros y bajo acoplamiento.
Documentación y entrega	10%	README, instrucciones reproducibles, decisiones técnicas y estructura del repositorio.

10.1 Extras valorados

- Uso de patrón Result, Specification, Mediator, Repository o Unit of Work cuando esté justificado.
- Manejo de eventos de dominio o mecanismo desacoplado para notificaciones.
- Contenedores Docker para backend, frontend o base de datos.
- Pruebas de integración para endpoints principales.
- Pipeline básico de CI o script de validación local.
- Semillas de datos representativas para facilitar la revisión.

11. Preguntas de conceptualización

1. ¿Cómo organizarías una solución .NET usando Clean Architecture o arquitectura por capas? Explica responsabilidades por proyecto o carpeta.
2. ¿Qué validaciones aplicarías en backend aun cuando el frontend ya valide formularios?
3. ¿Cómo diseñarías el mecanismo de notificaciones para permitir cambiar de correo a cola de mensajería sin reescribir la lógica de negocio?
4. ¿Qué estrategia usarías para evitar exponer datos de solicitudes a usuarios sin permisos?
5. ¿Cómo implementarías paginación, filtros y ordenamiento sin afectar el rendimiento?
6. ¿Qué diferencia hay entre pruebas unitarias, pruebas de integración y pruebas end-to-end? ¿Dónde aplicarías cada una en esta solución?
7. ¿Cuándo elegirías Entity Framework Core y cuándo considerarías Dapper?
8. ¿Cómo manejarías errores de forma consistente en una API REST?
9. ¿Qué información incluirías en logs sin comprometer datos sensibles?
10. ¿Cómo documentarías decisiones técnicas relevantes para facilitar mantenimiento futuro?

Criterios para estas respuestas:

- Claridad técnica y profundidad proporcional a la experiencia declarada.
- Capacidad de justificar decisiones y reconocer compromisos técnicos.
- Coherencia entre las respuestas conceptuales y la implementación entregada.

Uso exclusivo de la Superintendencia de Bancos. Distribución autorizada por el área responsable.

- Enfoque en seguridad, mantenibilidad, trazabilidad y calidad.

12. Forma de entrega

12.1 Aplicación

- Subir el código a un repositorio Git público o privado con acceso para revisión.
- Incluir README con instrucciones claras para ejecutar backend, frontend y base de datos.
- Incluir credenciales de prueba no sensibles o usuarios semilla documentados.
- El backend debe ejecutar localmente con .NET 8.
- El frontend debe ejecutar localmente con React y TypeScript.
- La solución debe incluir configuración de ambiente de ejemplo, sin secretos reales.
- Si no se usa Git, entregar archivo .zip con estructura completa y documentación.

12.2 Base de datos

- Entregar script .sql de creación de tablas y carga de datos de ejemplo, o migraciones de Entity Framework Core.
- Incluir instrucciones para aplicar migraciones o ejecutar scripts.
- Documentar cualquier supuesto relevante sobre motor de base de datos, versión o configuración.

12.3 Evidencias mínimas

- Capturas o breve guía del flujo principal.
- Archivo README completo.
- Instrucciones de autenticación y usuarios de prueba.
- Descripción de arquitectura y decisiones principales.
- Listado de pruebas incluidas y cómo ejecutarlas.

Anexo A. Recomendación de estructura de repositorio

```
/src
  /backend
    /Api
    /Application
    /Domain
    /Infrastructure
    /Tests
  /frontend
    /src
      /components
      /pages
      /services
      /types
      /hooks
/database
  schema.sql
  seed.sql
/docs
  arquitectura.md
  evidencias.md
README.md
```

Uso exclusivo de la Superintendencia de Bancos. Distribución autorizada por el área responsable.